

8 - dars: Pipeline in ML, Manual Preprocessingda muammolar, Auto Pipeline, Auto Pipeline Amaliy va log bilan ishlash review.

Pipeline in ML

Dastlabki qism:

- Load data
- Data Preprocessing
- Load data → select - features → cleaning → encoding
- Har bir qism (module) alohida va birgalikda sam manoga ega (ishlaydi)
- Sodda
- Murakkab

Pipeline in ML: Amaliy

class DataPreprocessing:


```
def __init__(self, full_df: pd.DataFrame):
    self.df = full_df.copy() # keep original safe
    self.encoders = {} # store encoders for each column
```

Handling Missing Values

```
def handle_missing_values(self):
    """ Drop rows/columns with too many NAs and fill remaining ones """
    logging.info("Handling missing values.")
    threshold = 0.5 * len(self.df)
    self.df = self.df.dropna(axis=1, thresh=threshold)
    self.df = self.df.fillna(0)
    return self.df
```

Remove duplicates

```
""" Remove Duplicates Rows. """
def remove_duplicates(self):
    logging.info("Removing duplicates rows ...")
    before = self.df.shape[0]
    self.df = self.df.drop_duplicates()
    after = self.df.shape[0]
    logging.info(f"Removed {before - after} duplicate rows.")
    return self.df
```

Categorical Encoding

```
def encode_categorical(self):
    """ Apply sklearn LabelEncoder to categorical columns """
    logging.info("Encoding categorical variables with LabelEncoders...")
    cat_cols = self.df.select_dtypes(include=["object"]).columns
```



```

for col in cat_cols:
    le = LabelEncoder()
    self.df[col] = le.fit_transform(self.df[col].
                                   astype(str))

    self.encoders[col] = le
    logging.info(f"Encoded columns: {col}")
return self.df

```

Scaling

```

def scale_numeric(self):
    """Scale numeric columns between 0 and 1 using MinMaxScaler."""
    logging.info("Scaling numeric columns with MinMaxScaler")
    num_cols = self.df.select_dtypes(include=["int64",
                                              "float64"]).columns.drop("target")
    if len(num_cols) > 0:
        self.scaler = MinMaxScaler()
        self.df[num_cols] = self.scaler.fit_transform(self.df[num_cols])
        logging.info(f"Scaled columns: {list(num_cols)}")
    else:
        logging.warning("No numeric columns found to scale.")
    return self.df

```

Final df ni tayyor berish uchun

```

def get_processed_data(self):
    """Return the final processed dataframe."""
    return self.df

```


`example.ipynb` ga ishlatis ketish uchun

Manual Pipeline

```
import os, sys
```

```
sys.path.append(r"copy path of source folder")
```

```
from pipeline import DataLoader, DataPreProcessing
```

```
loader = DataLoader(r"copy path of datasets stored")
```

```
try:
```

```
    full_df = loader.load_datasets()
```

```
    logging.info(f"Concat qilingan dataset class yordamida yuklandi {full_df.info()}")
```

```
except Exception as e:
```

```
    logging.error(f"Xatolik bor")
```

Preprocessing chapirish

```
preprocessing = DataPreProcessing(full_df)
```

```
try:
```

```
    df_clean = preprocessing.handle_missing_values()
```

```
    logging.info(f"Tushirib qoldirilgan qiymatlar to'ldirildi {df_clean.isnull().sum()}")
```

```
except Exception as e:
```

```
    logging.error(f"Xatolik bor")
```

duplicated columnlarni drop qilishni chaqirtish

try :

```
df_free_duplicated = preprocessing.remove_duplicated()
logging.info(f"Duplicated bilgani element / tashlab yutordiki: {df_free_duplicated.info()}")
```

except Exception as e:

```
logging.error(f"Xatolik bor")
```

Categorical Encodingni ham chaqiramiz.

try :

```
df_encoded = preprocessing.encode_categorical()
logging.info(f"Datasetimiz Cencoding qilindi {df_encoded.info()}")
```

except Exception as e:

```
logging.error(f"Xatolik bor")
```

Scaling qilishni chaqirish

try :

```
df_scaled = preprocessing.scale_numeric()
logging.info(f"Datasetimiz scaled qilindi {df_scaled.head()}")
```

except Exception as e:

```
logging.error(f"Xatolik bor")
```


final datasetni olishni chiqirish

try :

```
df_final = preprocessing.get_preprocessed_data()
logging.info(f"Final dataset {df_final}")
```

except Exception as e:

```
logging.error(f"Xatolik bor")
```

Manual Preprocessingni muammolar

• Manual Pipeline Approach:

↳ Multiple sequential steps # ko'plab bir-biriga bog'langan qadamlar

↳ Handle missing values

↳ Encode categorical features

↳ Scale numeric features

↳ Remove duplicates

↳ Train the model

• Issues:

↳ Tedious for large datasets # Pipeline jarayonida katta hajmdagi ma'lumot/ni qo'lda qayta ishlash juda mashqatli va samarasiz

↳ Error-prone for new input data # yangi kiruvchi ma'lumot/ni qayta ishlashda xatolar ko'p uchrashi m-u.

↳ Hard to deploy in production # Hasiyiy ishchi tizimga o'rnatish va ishga tushurish qiyin

Auto Pipeline

- ColumnTransformer : # Massas kltubxona, beshchi/ni avtomatik birlashtirib beradi
- Barcha transformation avtomatik kechadi :
 - ↳ Numeric → Missing values, MinMaxScaler, StandardScaler
 - ↳ Categorical → Missing values, OneHot Encoder, LabelEncoder
- Asosiy + : # advantages
 - ↳ Production - ready # deployment u-n oson bo'ladi
 - ↳ Clean & maintainable # kodlari ham oson va toza bo'ladi
 - ↳ Automatic handling of new data inputs # yangi kiruvchi ma'lumot/ni qayta ishlashda xatolar ~~hopp~~ uchrasa ~~munim~~ uchramaydi.